ECE-C103 · Data Structures · Week 2

Sheet 2 · Algorithms and data

Teaching-assistant practice based on Lecture 1 and Lecture 2. Use Python 3.14. Work in `DSA/sheet02`.

> ابدأ بالـ PLAN في كشكولك، وبعدها اكتب وشغّل. في السكشن: سؤال 4 الأول، ثم سؤال 5 لوحدك. الأسئلة الباقية للمراجعة والتدريب.

Keep code and outputs. Submit by the channel and deadline your TA announces. No deadline is assumed here.

Q1 · Swap two values

Predict the output of the following program without running it. Then run it and explain why it fails to swap the values. Repair it with a temporary variable named `t`. Save your repair as `q01_swap.py`.

```
a = 7
b = 12
a = b
b = a
print(a, b)
```

After a correct swap, the expected output is:

```
12 7
```

> اشرح في أنهي سطر قيمة a القديمة ضاعت. ما تستخدمش اختصار التبديل في Python.

Q2 · Return the larger value

Write pseudocode, then a function `larger(a, b)`. Return the larger numeric value. Equal values are allowed. Leave printing to the caller. Save as `q02_larger.py`.

```
print(larger(7, 12))
print(larger(12, 7))
print(larger(7, 7))
print(larger(-5, -2))
```

Expected output:

```
12
12
7
-2
```

Explain why the equality case works. Use HAVE / WANT / TRY / STEPS / CHECK in a comment block.

Q3 · Index and value

For `readings = [8, 12, 5, 12]`, predict `len(readings)` and `readings[1]`. Explain why `readings[4]` is outside the list. Then use `for i in range(len(readings))` to print each index and value.

Save as `q03_traversal.py`. Expected traversal output:

```
0 8
1 12
2 5
3 12
```

Q4 · Count readings at least a limit

Write `count_at_least(readings, limit)`. Return the number of values greater than or equal to the limit. Count repeated readings separately. Do not change the list. Do not use `sum`, a comprehension or a built-in filtering shortcut. Save as `q04_count.py`.

Plan provided: HAVE readings and limit. WANT a count. TRY [8, 12, 5, 12], limit 10. STEPS start at zero, visit every index, increment for a match, return after the loop. CHECK empty, equality, no matches and repeated matches.

```
print(count_at_least([8, 12, 5, 12], 10))
print(count_at_least([], 10))
print(count_at_least([10], 10))
print(count_at_least([2, 3], 10))
print(count_at_least([12, 12], 10))
```

Expected output:

```
2
0
1
0
2
```

> كل قراءة تتعدّ لوحدها. at least تشمل المساواة. وضّح ليه return بعد الـ loop.

Q5 · Change the condition yourself

Write `count_below(readings, limit)`. Count values strictly below the limit. Write your own TRY, STEPS and CHECK. Save as `q05_below.py`.

```
print(count_below([8, 12, 5, 12], 10))
print(count_below([10], 10))
print(count_below([], 10))
print(count_below([-2, 0, 3], 0))
```

Expected output:

```
2
0
0
1
```

Explain the difference between “below” and “at most”. Add one test that catches a mistaken `<=`.

> سؤال 5 اختبار نقل الفكرة. اكتب الخطة الأول، وبعدين غيّر الشرط بسبب المطلوب الجديد.

Q6 · Count search comparisons

Use the linear-search rule from Week 1: check positions in order and stop at the first match. For `[8, 12, 5, 12]`, record the returned index and the number of equality comparisons for targets `8`, `5`, `12` and `99`.

1. Which cases take the fewest and most comparisons?
2. What happens for an empty list?
3. If a target appears exactly once and is equally likely at any of four positions, what is the average count?
4. Does doubling n necessarily double every individual search's work? Explain.

Write answers in your notebook. No new search code is required.

Q7 · Represent an algorithm

Choose one task. Write pseudocode and draw a flowchart with labelled decision arrows. Then check the stated examples by hand or with fixed Python inputs.

- **Triangle:** base 6 cm, height 4 cm. Calculate area = base × height / 2. State the output unit.
- **Profit/loss:** compare income with cost. Include profit, loss and break-even. Test income/cost pairs 1000/800, 700/800 and 800/800.

What changes in your diagram when a task needs a decision?

Q8 · Data-structure reasoning

1. Distinguish a data type, an ADT and a concrete representation. Give one example of each.
2. For arrivals A, B, C, which leaves first under LIFO? Under FIFO?
3. Why is a linked list linear even if its nodes are not physically adjacent?
4. Describe traversal, search, insertion, deletion, sorting and merging using small lists.
5. You are told “linked insertion is always faster”. What information is missing?
6. Explain why Θ does not mean “average case”.
7. With coin values 1, 3 and 4, compare greedy change for 6 with a better combination.

Core coding: Q1–Q5. Lecture review: Q6–Q8. Keep your files and your written explanations.